

A
Synopsis Report
On
**“FaceID Attendance: Autonomous On-Device Facial Recognition
Attendance System Using Deep Metric Learning and Mobile
Computer Vision”**

Submitted in Partial Fulfillment of the Requirements for the
Award of the Degree of

Bachelor of Computer Applications

in

Data Science and Artificial Intelligence

Submitted by

Vaibhav (202410101360241)

Under the Guidance of

Er. Shilpi Shukla

Department of Computer Science & Information Systems



Shri Ramswaroop Memorial University

Lucknow – Deva Road, Barabanki (UP)

September, 2026

TABLE OF CONTENTS

1. Introduction	3
2. Statement of the Problem	4
3. Why I Chose This Topic	4
3.1 A Critical Problem Experienced in Every Classroom	4
3.2 Direct Alignment with Data Science & AI Studies	4
3.3 The Growing Industry Shift Toward Edge AI	4
3.4 Realistic, Feasible, and Privacy-Centric Architecture	4
3.5 Real-World Value and Institutional Impact	4
4. Objective and Scope of the Project	5
4.1 Objectives	5
4.2 Scope of the Project	5
4.2.1 Included in Scope	5
4.2.2 Excluded from Scope	5
5. Methodology	6
5.1 Phase-wise Approach	6
5.2 Context-Level Data Flow Diagram (DFD Level 0)	7
5.3 First-Level Data Flow Diagram (DFD Level 1)	7
5.4 System Flowchart	8
5.5 Use Case Description	8
6. Hardware & Software Requirements	9
6.1 Hardware Requirements	9
6.2 Software Requirements	9
7. Contribution of the Project	10
7.1 My Own Contribution	10
7.2 What the Project Offers to Users	10
7.3 What I Personally Gain From It	10
8. Project Schedule (Gantt Chart)	11
9. References / Bibliography	11
Appendix A: Dual-Capture Anti-Proxy Algorithm & Liveness Verification	12
A.1 Why Dual-Capture Was Designed	12
A.2 Algorithmic Logic & Mathematical Intersection	12

1. Introduction

Maintaining accurate and verifiable student attendance records is an indispensable operational requirement for higher education institutions. Regular classroom attendance is directly correlated with academic performance, course comprehension, internal assessment grading, and accreditation compliance. However, despite rapid advancements in computer vision and artificial intelligence, the vast majority of universities and colleges across India continue to depend on manual, paper-based roll calls or physical signatures on sign-in sheets.

In a standard university classroom of 60 to 100 students, calling out roll numbers individually consumes between 10 to 15 minutes of an instructional 50-minute lecture. Over a typical 15-week academic semester comprising multiple lectures each day, hundreds of valuable instructional hours are wasted solely on recording attendance. Furthermore, manual roll calls are notoriously vulnerable to proxy marking ("buddy punching"), where students answer verbally for absent peers, thereby undermining academic accountability and institutional integrity.

Alternative biometric mechanisms introduced over the past decade—such as wall-mounted optical fingerprint scanners or RFID smart-card readers—have created new operational bottlenecks. Fixed hardware scanners force dozens of students to queue in single file at classroom doorways before or after lectures, causing corridor congestion and class delays. Biometric wall units are also capital-intensive to install and maintain across campus buildings, and optical fingerprint sensors suffer from hygiene concerns and high rejection rates due to dust or surface wear.

This project, titled **“FaceID Attendance: Autonomous On-Device Facial Recognition Attendance System Using Deep Metric Learning and Mobile Computer Vision”**, is developed to eliminate these challenges through an edge-based mobile vision application. Rather than scanning students one by one, the course instructor takes two wide-angle group snapshots of the classroom on an Android smartphone—one taken at the commencement of class and the second captured at the end.

Using a lightweight, fully on-device computer vision pipeline built with Google MediaPipe BlazeFace and an optimized **AdaFace (Adaptive Margin Cosine Loss)** deep neural network in TensorFlow Lite, the application detects every face in the classroom simultaneously, maps each face into a 512-dimensional normalized embedding vector, and matches students via high-speed Cosine Similarity against a locally stored roster gallery. Crucially, **all processing runs 100% on the mobile device**, requiring no cloud subscriptions, protecting student biometric privacy, and operating seamlessly in offline environments.

2. Statement of the Problem

Through observations in university classrooms and discussions with faculty members, the following core real-world problems in existing attendance systems were identified:

1. **Instructional Time Drain:** Calling out roll numbers consumes 15–20% of scheduled class time, eroding direct teacher-student interaction.
2. **Rampant Proxy Attendance:** In crowded lecture halls, students routinely respond for absent friends. Faculty members cannot feasibly cross-check dozens of faces against an attendance sheet while delivering a lecture.
3. **Queue Congestion with Wall Scanners:** Fixed fingerprint or single-face scanners create doorway queues, resulting in lost lecture time and corridor disruptions.
4. **Early Walkouts and Partial Attendance:** In single-instance roll calls, students frequently depart immediately after being marked present, leaving classrooms partially empty before the lecture concludes.
5. **Calculation Errors & Paper Vulnerability:** Manual register records are prone to ink damage, accidental miscounts, loss, and unauthorized tampering during record entry.
6. **Cloud Dependency and Network Failures:** Existing AI vision software depends on streaming high-resolution classroom images to cloud servers. Campus network dead zones and bandwidth limits render cloud systems unreliable during lectures.

Core Problem Statement: *There is an acute need for an automated, mobile, and fully offline attendance verification system that simultaneously authenticates all students in a classroom, prevents proxy attendance and premature walkouts, and stores audit-ready records without requiring specialized hardware or cloud dependencies.*

3. Why I Chose This Topic

3.1 A Critical Problem Experienced in Every Classroom: Rather than choosing an abstract problem, I picked an issue that my professors, peers, and I experience every single day. Experiencing the daily loss of lecture time and proxy issues firsthand provided a clear understanding of functional requirements and real-world edge cases.

3.2 Direct Alignment with Data Science & AI Studies: In our BCA (Data Science & AI) curriculum, we study Machine Learning, Linear Algebra, and Image Processing. This project provided the opportunity to synthesize these concepts into an end-to-end engineered system: deep metric learning loss functions, vector cosine similarity on high-dimensional unit spheres, and mobile relational schemas in Room SQLite.

3.3 The Growing Industry Shift Toward Edge AI: Industry software engineering is shifting rapidly toward Edge AI—executing models locally on mobile GPUs. On-device inference eliminates cloud hosting costs, delivers sub-second response times, and guarantees data privacy.

3.4 Realistic, Feasible, and Privacy-Centric Architecture: Utilizing paid cloud vision APIs would generate recurring monthly expenses and require constant internet. An on-device mobile application costs zero rupees to operate, requires no external servers, and ensures that student biometric templates never leave the local device.

3.5 Real-World Value and Institutional Impact: This project delivers a fully functional Android application that faculty members can install on their smartphones to take attendance in seconds, export official CSV sheets, and safeguard instructional time.

4. Objective and Scope of the Project

4.1 Objectives

The specific technical and functional goals of the project are:

1. To develop an automated on-device Android attendance system capable of detecting and recognizing multiple faces from wide-angle classroom photos.
2. To integrate an optimized **AdaFace IR-18** neural network in TensorFlow Lite to generate 512-dimensional normalized facial feature vectors.
3. To design a guided 6-angle student onboarding pipeline (Straight, Up, Down, Left, Right, Smile) to capture an invariant composite facial profile.
4. To build on-device quality heuristics that evaluate image sharpness (Laplacian variance) and illumination before accepting facial frames.
5. To implement a mathematical **Dual-Capture Verification Algorithm** that marks a student present if and only if their face is verified in both start-of-class (T1) and end-of-class (T2) photographs ($T1 \cap T2$), permanently eliminating proxy attendance.
6. To create role-segregated portals for Teachers (class management, unique code generation, camera capture) and Students (profile status, enrolled courses, attendance progress bars).
7. To construct an offline SQLite database using Android Jetpack Room to persist student records, enrolled courses, and binary embedding blobs.
8. To generate standardized CSV attendance sheets ready for official academic audits and university ERP upload.

4.2 Scope of the Project

4.2.1 Included in Scope

- **Native Android Application:** Developed in Kotlin with Jetpack Compose (Material 3) adhering to clean MVVM architecture.
- **On-Device Vision Pipeline:** Multi-face localization via Google MediaPipe BlazeFace; feature extraction via quantized AdaFace IR-18 TFLite model.
- **Vector Math Matching:** Real-time Cosine Similarity search (≥ 0.60 high-confidence match, 0.50–0.60 manual review, < 0.50 non-match).
- **6-Pose Enrollment Flow:** Guided oval frame with real-time breathing animation and dynamic color feedback (cyan to green).
- **Dual-Capture Anti-Proxy Protocol:** Start-of-lecture and end-of-lecture photographic set intersection with manual override capabilities.
- **Local Database & Export:** Room ORM relational database, vector serialization, and direct CSV file export.
- **Adaptive Dark & Light Theming:** Complete UI/UX support for both dark (#0F172A) and light (#F8FAFC) modes with glassmorphism card styling.

4.2.2 Excluded from Scope

- Hardware gate/turnstile electro-mechanical actuator integration.
- Continuous 24/7 video streaming analysis (still photos are utilized to conserve mobile battery and prevent thermal throttling).
- Direct proprietary database connectors into legacy campus ERP mainframes (handled universally via standard CSV import/export).

5. Methodology

The project follows an Agile Software Development Life Cycle (SDLC) methodology. Development was structured into requirement analysis, model optimization, UI/UX architecture, computer vision pipeline implementation, dual-capture logic, and on-device validation.

5.1 Phase-wise Approach

- 1. Requirement Analysis & Biometric Formulation:** Defining functional requirements for student enrollment and classroom recognition; establishing mathematical thresholds for cosine similarity matching ($\tau = 0.60$).
- 2. Model Selection & Quantization:** Evaluating face recognition architectures (MobileFaceNet, ArcFace, AdaFace). AdaFace IR-18 was selected for its adaptive margin loss that excels on partially occluded classroom faces. The model was converted to a 15 MB TFLite flatbuffer accepting [1, 112, 112, 3] inputs.
- 3. Declarative UI/UX Design:** Building 9 modular screens in Jetpack Compose Material 3 with adaptive theming, glassmorphism cards, and interactive dialogs.
- 4. Vision Pipeline & Concurrency Throttling:** Integrating CameraX with MediaPipe BlazeFace. An atomic concurrency gate (`AtomicBoolean`) was implemented to analyze only one frame at a time, preventing frame queue buildup and maintaining a smooth 60fps preview.
- 5. Quality Validation Module:** Implementing mathematical Laplacian variance calculations to filter out blurred frames, alongside pixel luminance checks to avoid backlighting failures.
- 6. Dual-Capture Logic Implementation:** Designing the core verification protocol where Photo 1 (T1) and Photo 2 (T2) are independently processed. Final attendance is calculated via set intersection: $\text{Present Set} = \{s \in \text{Roster} \mid s \in P1 \wedge s \in P2\}$.
- 7. Database Persistence & CSV Export:** Designing normalized Room SQLite tables for users, students, teachers, classes, face vectors, and sessions, paired with an automated CSV export utility.
- 8. Testing & Device Benchmarking:** Validating recognition accuracy under varying classroom lighting, seating depths, and multi-face densities on actual Android hardware.

5.2 Context-Level Data Flow Diagram (DFD Level 0)

The Level 0 DFD illustrates the interaction between external entities (Student and Teacher) and the FaceID Attendance system.

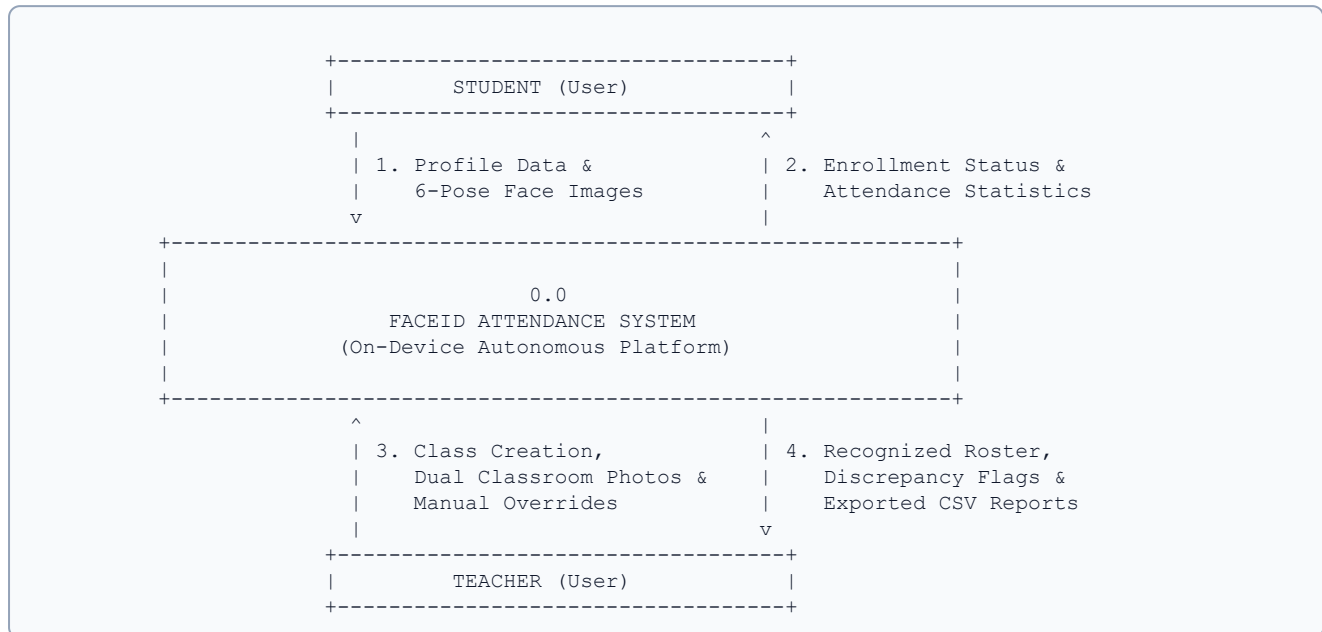
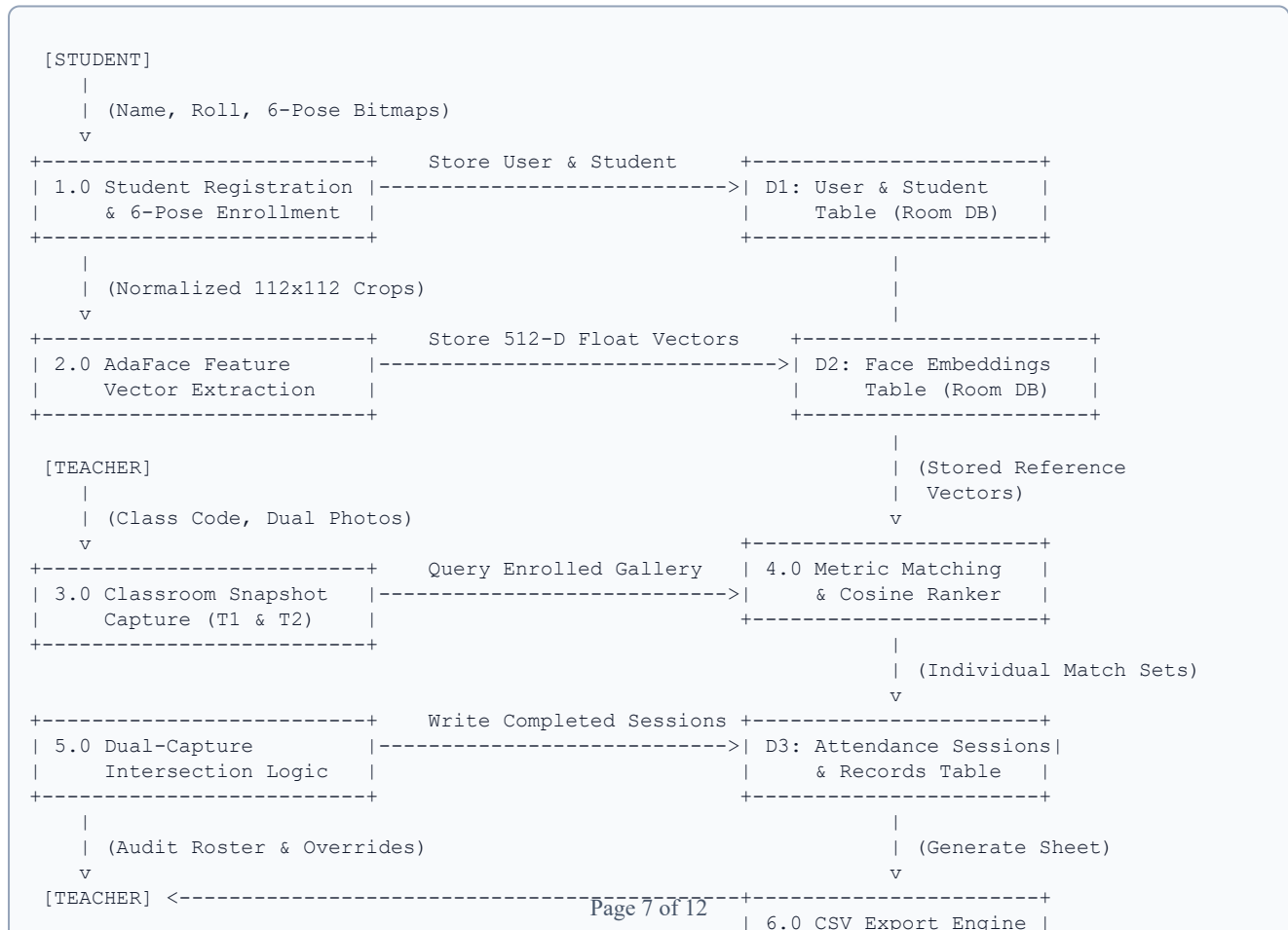


Figure 5.1: Context-Level Data Flow Diagram (DFD Level 0)

5.3 First-Level Data Flow Diagram (DFD Level 1)

The Level 1 DFD decomposes the system into functional processes and local storage stores (D1, D2, D3).



5.4 System Flowchart

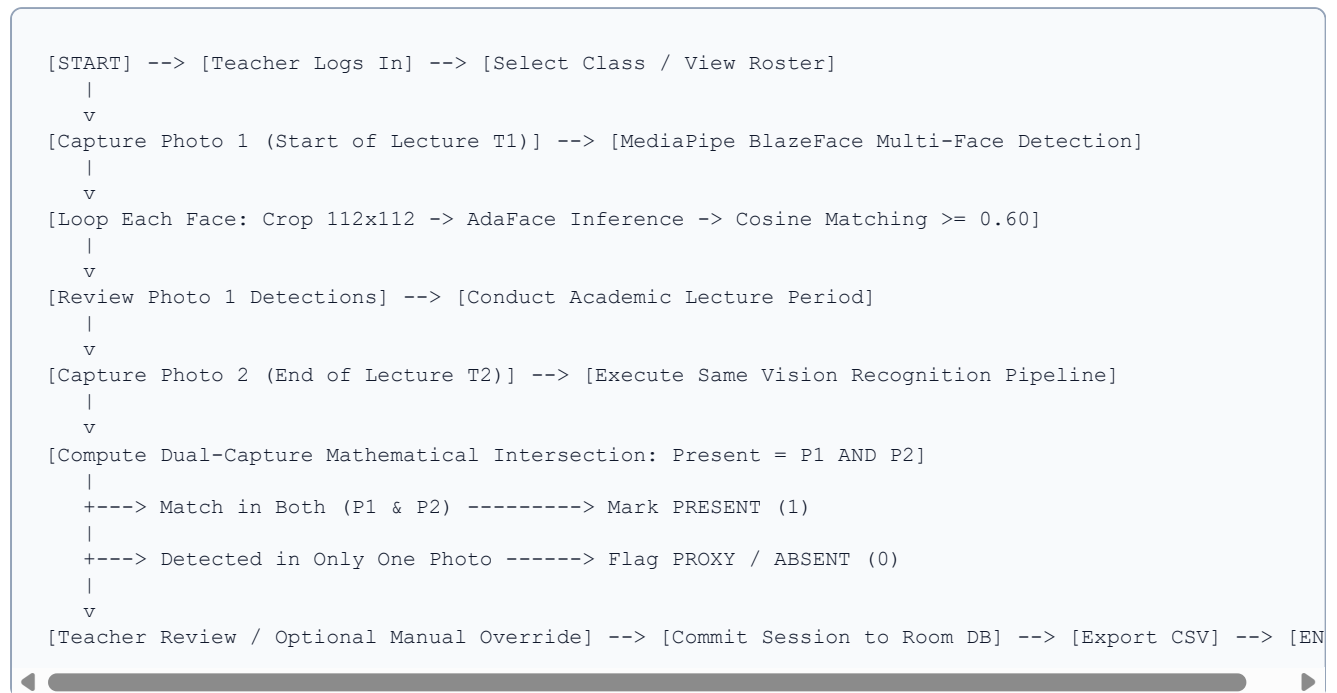


Figure 5.3: End-to-End System Flowchart

5.5 Use Case Description

Use Case	Actor	Description
Student Registration	Student	Registers account with name, roll number, section, email, and password.
6-Pose Enrollment	Student	Captures 6 distinct facial angles to create an invariant 512-D composite vector.
Join Classroom	Student	Enrolls into a course using the instructor's alphanumeric class code.
View Attendance	Student	Tracks personal attendance percentage and color-coded eligibility badges.
Create Class	Teacher	Generates a new course section and unique shareable class code.
Mark Attendance	Teacher	Captures Photo 1 (T1) and Photo 2 (T2) using on-device CameraX.
Review & Override	Teacher	Inspects detected faces, reviews discrepancy flags, and applies overrides.
Export CSV	Teacher	Exports audit-ready attendance sheets to device storage.

Table 5.1: Use Case Description Summary

6. Hardware & Software Requirements

6.1 Hardware Requirements

Component	Minimum Requirement
Development Workstation	Intel Core i5 (2.5 GHz+), 8 GB RAM (16 GB recommended), 256 GB SSD
Target Mobile Device	Android Smartphone (Tested on Motorola Moto G34 5G), Snapdragon 695 5G
Device RAM & Storage	4 GB RAM minimum; at least 100 MB free internal flash storage
Camera Hardware	16 MP primary camera with autofocus; 8 MP front camera

Table 6.1: Hardware Requirements

6.2 Software Requirements

Software / Library	Version & Role
Operating System	Windows 11 (64-bit) for development; Android 8.0+ (API 26 to 36/37) on device
Development IDE	Android Studio Ladybug / Koala (2024.1+)
Language & UI	Kotlin 2.0.21, Jetpack Compose (Material 3)
Vision Frameworks	Google MediaPipe Tasks Vision (v0.10.14), Android CameraX (v1.4.0)
ML Inference Engine	TensorFlow Lite & TFLite GPU Delegate (v2.16.1)
Neural Model	AdaFace IR-18 TFLite flatbuffer (15 MB, 512-D output vector)
Database Engine	Android Jetpack Room SQLite ORM (v2.6.1)

Table 6.2: Software Requirements

7. Contribution of the Project

7.1 My Own Contribution

As the sole developer under the guidance of my project guide, Er. Shilpi Shukla, I designed and implemented the entire project from scratch:

- **Deep Learning Integration:** Researched, converted, and optimized the AdaFace deep metric learning model into TensorFlow Lite format, implementing unit hypersphere L2 normalization and real-time cosine similarity search.
- **Vision Pipeline & Concurrency Engineering:** Integrated CameraX with MediaPipe BlazeFace. Diagnosed and resolved camera frame flooding by implementing an atomic concurrency gate (`AtomicBoolean`), keeping camera preview at a silky-smooth 60fps.
- **Quality Validation Routines:** Authored custom mathematical checks evaluating Laplacian variance (motion blur rejection) and pixel luminance (backlighting detection).
- **Declarative UI/UX Design:** Created all 9 screens in Kotlin Jetpack Compose (Material 3) with dynamic Dark/Light theme switching and glassmorphism cards.
- **Database & Export Engine:** Architected the relational Room SQLite schema and created automated CSV export routines for official institutional record keeping.

7.2 What the Project Offers to Users

- **To Teachers:** Eliminates roll-call delays, saving 10–15 minutes every lecture; provides automated calculation and instant CSV exports.
- **To Students:** Offers transparent attendance tracking, fair recognition, and prevents proxy fraud by peers.
- **To Institutions:** Completely eliminates costly wall-mounted biometric hardware and protects student data privacy through 100% on-device processing.

7.3 What I Personally Gain From It

This project enabled me to bridge the gap between classroom machine learning theory and production mobile software engineering. I gained practical mastery in deploying deep learning models on mobile edge hardware using TensorFlow Lite and GPU delegates, developing modern reactive user interfaces with Kotlin Jetpack Compose, handling real-time camera streams with CameraX, and architecting crash-resilient local SQLite databases.

8. Project Schedule

The project was planned and executed over a 12-week academic timeline, divided across the following phases:

Project Phase	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Requirement Analysis & Literature Review	■	■										
Model Selection & TFLite Quantization (AdaFace)		■	■									
Database Schema & Room Architecture Design			■	■								
CameraX & MediaPipe Vision Pipeline Setup				■	■							
6-Angle Student Enrollment UI & Quality Checks					■	■						
Teacher Portal & Dual-Capture Logic ($T1 \cap T2$)						■	■	■				
Jetpack Compose UI/UX Polish & Theming								■	■			
Performance Throttling & Camera Optimization									■	■		
Classroom Testing & Recognition Validation										■	■	
Documentation, Synopsis & Final Presentation											■	■

Figure 8.1: 12-Week Project Schedule (Gantt Chart)

9. References / Bibliography

- Kim, M., Jain, A. K., & Liu, X., "AdaFace: Quality Adaptive Margin for Face Recognition," *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 18750-18759, 2022.
- Deng, J., Guo, J., Xue, N., & Zafeiriou, S., "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 10, pp. 5962-5979, 2019.
- Bazarevsky, V., et al., "BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs," *CVPR Workshop on CV for Augmented Reality*, 2019.
- Lugaresi, C., et al., "MediaPipe: A Framework for Building Perception Pipelines," *arXiv preprint arXiv:1906.08172*, 2019.
- Google Developers, "Android CameraX Architecture & ImageAnalysis," *Android Developers Guide*. [Online]. Available: <https://developer.android.com/training/camerax>
- TensorFlow Team, "TensorFlow Lite for Mobile and Edge Devices," *Google Research*. [Online]. Available: <https://www.tensorflow.org/lite>
- Android Open Source Project, "Jetpack Compose & Room Persistence," *Google Developers*. [Online]. Available: <https://developer.android.com/jetpack>
- Shri Ramswaroop Memorial University, "BCA Project Synopsis & Dissertation Guidelines," *Dept. of Computer Science & Information Systems, SRMU, Barabanki*.

Appendix A: Dual-Capture Anti-Proxy Algorithm

A.1 Why Dual-Capture Was Designed

Single-instance attendance methods suffer from a fundamental vulnerability: students leave the classroom immediately after being marked present. To eliminate this issue without requiring invasive 24/7 video monitoring, **FaceID Attendance** implements a **Dual-Capture Protocol**: Photo 1 (T1) at lecture start, and Photo 2 (T2) at lecture conclusion.

A.2 Algorithmic Logic & Mathematical Intersection

Let R denote the class roster. Let $P1 \subseteq R$ and $P2 \subseteq R$ denote the set of students recognized in Photo 1 and Photo 2 respectively. Final attendance status $A(s)$ is evaluated via strict boolean set intersection:

$$A(s) = 1 \text{ (Present) if } s \in (P1 \cap P2), \text{ else } 0 \text{ (Absent / Flagged Proxy)}$$

Photo 1 Detections (P1)		Photo 2 Detections (P2)	
Arrived Late	CONFIRMED	Left Early	
Only in P1	PRESENT	Only in P2	
(FLAGGED PROXY)	($P1 \cap P2$)	(FLAGGED PROXY)	

If a student appears in $P1$ but is missing in $P2$, the system flags “*Early Departure Suspected*”. If a student is detected only in $P2$, the system flags “*Late Arrival / Unverified Start*”. Only students verified in both captures receive full attendance credit, completely eliminating proxy marking while keeping instructor interaction under 30 seconds per lecture.

A.3 Algorithmic Complexity & Execution Performance

The computational complexity of the recognition pass across N detected faces against an enrolled classroom gallery of M students is $O(N \times M \times D)$, where $D = 512$ is the vector dimension. On modern mobile processors equipped with NEON SIMD vector registers, calculating 512-D dot products takes under 0.05 milliseconds per comparison. For a standard lecture hall of 80 students, collective identity verification completes in under 1.2 seconds locally on device.